

Scaling Book — Section 10 Exercises

Rahul Bir

q1

part 1 (computing averages of each shard):

```
1 import jax
2 import jax.numpy as jnp
3 Auto = jax.sharding.AxisType.Auto
4 import functools
5 import numpy as np
6 # jax.config.update('jax_num_cpu_devices', 8)
```

```
1 # method 1: implemented with jax.jit
2 mesh = jax.make_mesh((4, 2), ('X', 'Y'), (Auto, Auto))
3 jax.set_mesh(mesh)
4
5 A = jnp.arange(32 * 64).reshape((32, 64))
6 A = jax.device_put(A, jax.P('X', 'Y'))
7
8
9 @functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
10 def shard_avg(A):
11     r, c = jnp.shape(A)
12     r_local, c_local = r // 4, c // 2
13     return jnp.array([
14         jnp.mean(A[r_local * i:r_local * i + r_local,
15                   c_local * j:c_local * j + c_local],
16                 axis=None)
17         for i in range(4)
18         for j in range(2)
19     ]).reshape((4, 2))
20
21 with jax.profiler.trace("/kaggle/working/q1_p1_tensorboard_jit"):
22     out_jit = shard_avg(A).block_until_ready()
23
24 print(out_jit)
```

```
1 # method 2: implemented with shard_map
2 mesh = jax.make_mesh((4, 2), ('X', 'Y'))
3 jax.set_mesh(mesh)
4
5 A = jnp.arange(32 * 64).reshape((32, 64))
6 A = jax.device_put(A, jax.P('X', 'Y'))
7
8 @jax.jit
9 @jax.shard_map(in_specs=jax.P('X', 'Y'), out_specs=jax.P('X', 'Y'))
10 def shard_avg_shm(A):
```

```

11     return jnp.array([[jnp.mean(A)]])
12
13 with jax.profiler.trace("/kaggle/working/q1_p1_tensorboard_shm"):
14     out_shm = shard_avg_shm(A).block_until_ready()
15
16 print(out_shm)

```

```

1 print(jnp.dtype(out_jit))
2 print(jnp.dtype(out_shm))
3 np.testing.assert_allclose(out_jit, out_shm)

```

The jit version takes around 65.7 micro seconds to complete whereas the shardmap version takes only 651.3 nano seconds. I was very surprised by the orders of magnitude difference! Another interesting thing is that the jit version added many communication primitives even though it needed to only run 1 fusion (which the shard map version does).

part 2 (implementing roll - x for each shard):

```

1 mesh = jax.make_mesh((4, 2), ('X', 'Y'))
2 jax.set_mesh(mesh)
3
4 A = jnp.arange(32 * 64).reshape((32, 64))
5 A = jax.device_put(A, jax.P('X', 'Y'))
6
7
8 def roll_rows_sub(x, shift):
9     return jax.jit(jax.shard_map(
10         lambda x: jnp.roll(x, shift=shift, axis=0) - x,
11         in_specs=jax.P('X', 'Y'),
12         out_specs=jax.P('X', 'Y')
13     ))(x)
14
15 out = roll_rows_sub(A, 2)
16 print(out)
17
18 out = roll_rows_sub(A, 0)
19 ref = jnp.zeros((32, 64), dtype=jnp.int32, device=jax.NamedSharding(mesh, jax.P('X', 'Y')))
20 np.testing.assert_array_equal(out, ref)

```

q2

part 1:

```

1 """
2 want to avoid materializing [S, D, F] array (very large!)
3
4 instead:
5
6 1) rearrange [S, D] sequence matrix into shape [E, S, D].
7 (Note: We need the array to be size [E, S, D] because normal jax arrays can't be
8    ragged
9    even though each expert has <= S tokens routed to it)
10
11 2) do matmul with W

```

```

11
12 3) index out based on correct experts
13 """
14
15 def moe_local(A, W, B):
16     return jnp.einsum('edf,esd->esf',
17                       W,
18                       jnp.expand_dims(A, 0)
19                       )[B, jnp.arange(S)]

```

part 2:

```

1 S = 8
2 D = 4
3 F = 32
4 E = 4
5
6 Auto = jax.sharding.AxisType.Auto
7 mesh = jax.make_mesh((1,), ('X',), (Auto,))
8 jax.set_mesh(mesh)
9
10 @jax.jit
11 def init_arrays(key1, key2, key3):
12     W = jax.random.normal(key1,
13                           (E, D, F),
14                           dtype=jnp.float32)
15
16     A = jax.random.normal(key2,
17                           (S, D),
18                           dtype=jnp.float32)
19
20     B = jax.random.randint(key3,
21                            (S,),
22                            dtype=jnp.int32,
23                            minval=0,
24                            maxval=E)
25
26     W = jax.device_put(W, jax.P('X', None, None))
27     A = jax.device_put(A, jax.P('X', None))
28     B = jax.device_put(B, jax.P('X'))
29
30     return W, A, B
31
32
33 key1 = jax.random.key(42)
34 key2, _ = jax.random.split(key1)
35 key3, _ = jax.random.split(key2)
36
37
38 moe_jit = jax.jit(moe_local)
39 W, A, B = init_arrays(key1, key2, key3)
40
41 with jax.profiler.trace("/kaggle/working/q2_p2_tensorboard_naive_jit"):
42     out_jit = moe_jit(A, W, B).block_until_ready()
43
44 print('output out_jit:')
45 print(out_jit)

```

part 3:

```
1 import numpy as np
2
3 """
4 pass 1 ideas:
5 AG_X(W[E@X, D, F]) = W[E, D, F]
6 W[E, D, F] @_D A[E, S@X, D] => [E, S@X, D]
7 Index correct values based on B[S@X]
8 """
9
10 @jax.jit
11 @jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X')),
12               out_specs=jax.P('X', None))
13 def moe_shm(A, W, B):
14     W = jax.lax.all_gather(W, 'X', tiled=True)
15     return jnp.einsum('edf,esd->esf',
16                      W,
17                      jnp.expand_dims(A, 0)
18                      )[B, jnp.arange(S // 8)]
19
20 out_shm = moe_shm(A, W, B)
21 out_jit = moe_jit(A, W, B)
22 print(jax.typeof(out_shm))
23
24 np.testing.assert_allclose(out_jit, out_shm)
25
26
27 """
28 inefficiencies:
29 1) need to materialize W[E, D, F] array on each device
30 2) wasteful computation on "padded" [E, S, D] array (need to make it ragged)
31 """
```

```
1 """
2 pass 2:
3
4 main ideas:
5 1) create ragged array to minimize empty padding flops
6 2) get rid of big AG and replace with cheaper AllToAlls
7 """
8
9 @jax.jit
10 @jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X')),
11               out_specs=jax.P('X', None))
12 def moe_shm_chunked(A, W, B):
13     sorted_B_inds = jnp.argsort(B)
14     sorted_B = B[sorted_B_inds]
15     expert_sorted_A = A[sorted_B_inds]
16     E = W.shape[0] * jax.lax.axis_size('X')
17     sizes = jnp.bincount(B, minlength=E, length=E)
18     global_max_expert_size = jnp.max(jax.lax.pmax(sizes, 'X'))
19     start_inds = jnp.cumsum(sizes) - sizes
20
21     CHUNK_SIZE = 8
22     def get_chunk(i):
23         inds = (CHUNK_SIZE * i) + start_inds[:, None] + jnp.arange(CHUNK_SIZE)[
```

```

24         None, :]
25     chunk = expert_sorted_A[jnp.ravel(inds)].reshape((E, CHUNK_SIZE, A.shape
26         [1]))
27     end_inds = jnp.append(jax.lax.dynamic_slice_in_dim(start_inds, 1, E-1),
28         jnp.array([A.shape[0]]))
29     valid = inds < end_inds[:, None]
30     return chunk, valid, inds
31
32 def f(i, out):
33     chunk, valid, inds = get_chunk(i)
34     chunk = jax.lax.all_to_all(chunk, 'X', split_axis=0, concat_axis=1, tiled=
35         True)
36     chunk = jnp.einsum('ecd,edf->ecf', chunk, W)
37     chunk = jax.lax.all_to_all(chunk, 'X', split_axis=1, concat_axis=0, tiled=
38         True)
39     # setting values later for the update is nondeterministic, so write
40     # invalid inds to a padding row instead
41     masked_inds = jnp.where(valid, inds, -1)
42     flattened_inds = jnp.ravel(masked_inds)
43     flattened_chunk = chunk.reshape(E * CHUNK_SIZE, W.shape[2])
44     flattened_valid = jnp.ravel(valid)[:, None]
45     update = jnp.where(flattened_valid, flattened_chunk, 0)
46     new_out = out.at[flattened_inds].set(update)
47     return new_out
48
49 # add padding rows so writes on padding don't overwrite real values
50 out = jnp.zeros((A.shape[0] + 1, W.shape[2]))
51 out = jax.lax.pcast(out, 'X', to='varying')
52 out = jax.lax.fori_loop(0, (global_max_expert_size + CHUNK_SIZE - 1) //
53     CHUNK_SIZE, f, out)
54
55 return out[jnp.argsort(sorted_B_inds)]

```

```

1 # run comparison test for thoroughness
2 Auto = jax.sharding.AxisType.Auto
3 Explicit = jax.sharding.AxisType.Explicit
4
5
6 def run_test(S, D, F, E, seed):
7     with jax.set_mesh(jax.make_mesh((8,), ('X'), (Auto,))):
8         key1 = jax.random.key(seed)
9         key2, key3 = jax.random.split(key1)
10        W = 10 * jax.random.normal(key1,
11            (E, D, F),
12            dtype=jnp.float32)
13        A = 10 * jax.random.normal(key2,
14            (S, D),
15            dtype=jnp.float32)
16        B = jax.random.randint(key3,
17            (S,),
18            dtype=jnp.int32,
19            minval=0,
20            maxval=E)
21        W = jax.device_put(W, jax.P('X', None, None))
22        A = jax.device_put(A, jax.P('X', None))
23        B = jax.device_put(B, jax.P('X'))
24
25        moe_jit = jax.jit(moe_local)

```

```

26     with jax.profiler.trace("/kaggle/working/q2_p2_tensorboard_naive_jit"):
27         out_jit = moe_jit(A, W, B).block_until_ready()
28
29     with jax.set_mesh(jax.make_mesh((8,), ('X',)), (Explicit,)):
30         key1 = jax.random.key(seed)
31         key2, key3 = jax.random.split(key1)
32         W = 10 * jax.random.normal(key1,
33                                     (E, D, F),
34                                     dtype=jnp.float32,
35                                     out_sharding=jax.P('X', None, None))
36         A = 10 * jax.random.normal(key2,
37                                     (S, D),
38                                     dtype=jnp.float32,
39                                     out_sharding=jax.P('X', None))
40         B = jax.random.randint(key3,
41                                 (S,),
42                                 dtype=jnp.int32,
43                                 minval=0,
44                                 maxval=E,
45                                 out_sharding=jax.P('X'))
46
47     with jax.profiler.trace("/kaggle/working/q2_p3_tensorboard_shm_chunked"):
48         out_shm_chunked = moe_shm_chunked(A, W, B).block_until_ready()
49
50
51     np.testing.assert_allclose(out_shm_chunked, out_jit, rtol=1e-5, atol=1e-5)
52     print("TEST PASSED")
53
54
55
56     S = 512
57     D = 4096
58     F = 4096*4
59     E = 8
60
61     """
62     With 16 experts, and 512 tokens.
63
64     Want to minimize padding. In best case scenario
65     tokens are balanced fully across each expert,
66     so each expert gets 512/16 = 32 tokens routed
67     to them. => choosing a CS=8 seems reasonable.
68     """
69
70     run_test(S, D, F, E, 42)

```

The naive jit kernel takes 2.665 ms, whereas the chunked moe shardmap version I wrote above takes 1.669 ms! This is under the S, D, F, E values above on a v5e-8 cluster.

The roofline for this is $2 \cdot S \cdot D \cdot F / (8 \cdot C) = 0.08$ ms.

part 4

```

1     """
2     route to (k) experts and average the result
3
4     W[E@X, D, F]
5     A[S@X, D]

```

```

6 B[S@X, k]
7
8 qs:
9
10 - how to fill chunks? -> need to map the same token to multiple experts
11
12
13 core idea:
14
15 reconstruct into S*k out, keep logic the same, add extra k dimension when writing
    back?
16 """
17 @jax.jit
18 def moe_k_experts_naive(A, W, B):
19     """
20     W[E@X, D, F]
21     A[S@X, D]
22     B[S@X, k]
23     """
24     out = []
25     for s_i in range(A.shape[0]):
26         token = A[s_i] # [D]
27         experts = B[s_i] # [k]
28         selected_W = W[experts] # [k, D, F]
29         out.append(jnp.einsum('kdf,d->kf', selected_W, token).mean(axis=0))
30
31     return jnp.stack(out)
32
33
34 @jax.jit
35 @jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X', None
    )), out_specs=jax.P('X', None))
36 def moe_k_experts_shm(A, W, B):
37     """
38     B[S@X, k]
39
40     b_flat = B[S_x*k,]
41     b_flat_sorted = sort(B[S_x*k,]) => B[S_x*k,]
42     A_inds = jnp.arange
43
44     """
45
46     # jax.debug.print('ORIGINAL B: \n {b}', b=B, ordered=True)
47     flattened_B = B.flatten()
48     # jax.debug.print('FLATTENED B: {b}', b=flattened_B, ordered=True)
49     sorted_flat_B_inds = jnp.argsort(flattened_B)
50     sorted_flat_B = flattened_B[sorted_flat_B_inds]
51     # jax.debug.print('SORTED FLAT B inds: \n {b}', b=sorted_flat_B_inds, ordered=
        True)
52     # jax.debug.print('SORTED FLAT B: \n {b}', b=sorted_flat_B, ordered=True)
53     rep_flat_A_inds = jnp.repeat(
54         jnp.arange(A.shape[0])[:, None],
55         B.shape[1],
56         axis=-1,
57         total_repeat_length=B.shape[1]
58     ).flatten()
59     # jax.debug.print('repeated flat A inds: \n {b}', b=rep_flat_A_inds, ordered=
        True)
60     sorted_rep_flat_A_inds = rep_flat_A_inds[sorted_flat_B_inds]

```

```

61 # jax.debug.print('sorted repeated flat A inds: \n {b}', b=
    sorted_rep_flat_A_inds, ordered=True)
62 E = W.shape[0] * jax.lax.axis_size('X')
63 sizes = jnp.bincount(flattened_B, minlength=E, length=E)
64 # jax.debug.print('sizes: \n {s}', s=sizes, ordered=True)
65 global_max_expert_size = jnp.max(jax.lax.pmax(sizes, 'X'))
66 start_inds = jnp.cumsum(sizes) - sizes
67 # jax.debug.print('start_inds: \n {s}', s=start_inds, ordered=True)
68
69 CHUNK_SIZE = 16
70 def get_chunk(i):
71     """
72     inds is relative to the sorted_A_inds
73     """
74     # jax.debug.print('-----', ordered=True)
75     # jax.debug.print('getting chunk {i}', i=i, ordered=True)
76     # jax.debug.print('original A for reference: \n {a}', a=A, ordered=True)
77     # inds[E, CS]
78     inds = (CHUNK_SIZE * i) + start_inds[:, None] + jnp.arange(CHUNK_SIZE)[
        None, :]
79     # jax.debug.print('inds to get from sortedA: \n {a}', a=inds, ordered=True
    )
80     # jax.debug.print('corresponding original A inds: \n {i}', i=
        sorted_rep_flat_A_inds[inds.flatten()].reshape((E, CHUNK_SIZE)),
        ordered=True)
81     # chunk[]
82     chunk = A[sorted_rep_flat_A_inds[inds.flatten()]].reshape((E, CHUNK_SIZE,
        A.shape[1]))
83     # jax.debug.print('CHUNK: \n {c}', c=chunk, ordered=True)
84     end_inds = jnp.append(jax.lax.dynamic_slice_in_dim(start_inds, 1, E-1),
        jnp.array([A.shape[0] * B.shape[1]]))
85     valid = inds < end_inds[:, None]
86     # jax.debug.print('valid mask: \n {c}', c=valid, ordered=True)
87     # jax.debug.print('-----', ordered=True)
88     return chunk, valid, inds
89
90
91 def f(i, out):
92     # jax.debug.print('----- STARTING F {i}', i=i, ordered=True)
93     chunk, valid, inds = get_chunk(i)
94     # jax.debug.print('chunk received: \n {c}', c=chunk, ordered=True)
95     # jax.debug.print('valid received: \n {c}', c=valid, ordered=True)
96     # jax.debug.print('inds received: \n {c}', c=inds, ordered=True)
97     chunk = jax.lax.all_to_all(chunk, 'X', split_axis=0, concat_axis=1, tiled=
        True)
98     chunk = jnp.einsum('ecd,edf->ecf', chunk, W)
99     # jax.debug.print('after einsum chunk: \n {c}', c=chunk, ordered=True)
100    chunk = jax.lax.all_to_all(chunk, 'X', split_axis=1, concat_axis=0, tiled=
        True)
101    # setting values later for the update is nondeterministic, so write
        invalid inds to a padding row instead
102    masked_inds = jnp.where(valid, inds, -1)
103    # jax.debug.print('masked inds: \n {c}', c=masked_inds, ordered=True)
104    flattened_inds = jnp.ravel(masked_inds)
105    flattened_chunk = chunk.reshape(E * CHUNK_SIZE, W.shape[2])
106    flattened_valid = jnp.ravel(valid)[:, None]
107    update = jnp.where(flattened_valid, flattened_chunk, 0)
108    new_out = out.at[flattened_inds].set(update)
109    # jax.debug.print('new out: \n {n}', n=new_out, ordered=True)

```



```

110         return new_out
111
112
113     out = jnp.zeros((A.shape[0] * B.shape[1] + 1, W.shape[2]))
114     # out = jax.lax.pcast(out, 'X', to='varying')
115     out = jax.lax.pvary(out, 'X')
116     out = jax.lax.fori_loop(0, (global_max_expert_size + CHUNK_SIZE - 1) //
117                             CHUNK_SIZE, f, out)
118
119     out = out[jnp.argsort(sorted_flat_B_inds)].reshape((A.shape[0], B.shape[1], W.
120                                                         shape[2])).mean(axis=1)
121
122     return out
123
124
125 S = 512
126 D = 4096
127 F = 4096*4
128 E = 8
129 k = 4
130 seed = 42
131
132 """
133 512*4 = 2048 tokens to route + process
134 2048/8 = 256
135 => setting a CS = 16 seems reasonable
136 """
137
138 Explicit = jax.sharding.AxisType.Explicit
139 Auto = jax.sharding.AxisType.Auto
140
141 def run_test(S, D, F, E, k, seed):
142     def generate_B(seed, k, E, S):
143         out = []
144         key = jax.random.key(seed)
145         for _ in range(S):
146             key, subkey = jax.random.split(key)
147             out.append(jax.random.choice(subkey, E, (k,), replace=False))
148         return jnp.stack(out)
149
150     with jax.set_mesh(jax.make_mesh((8,), ('X',), (Explicit,))):
151         key1 = jax.random.key(seed)
152         key2, key3 = jax.random.split(key1)
153         W = 10 * jax.random.normal(key1,
154                                     (E, D, F),
155                                     dtype=jnp.float32,
156                                     out_sharding=jax.P('X', None, None))
157         A = 10 * jax.random.normal(key2,
158                                     (S, D),
159                                     dtype=jnp.float32,
160                                     out_sharding=jax.P('X', None))
161         B = generate_B(seed, k, E, S)
162         B = jax.device_put(B, jax.P('X', None))
163
164         out_shm_chunked = moe_k_experts_shm(A, W, B).block_until_ready()
165
166     with jax.set_mesh(jax.make_mesh((8,), ('X',), (Auto,))):
167         key1 = jax.random.key(seed)
168         key2, key3 = jax.random.split(key1)
169         W = 10 * jax.random.normal(key1,

```

```

167         (E, D, F),
168         dtype=jnp.float32)
169     A = 10 * jax.random.normal(key2,
170                                (S, D),
171                                dtype=jnp.float32)
172     B = generate_B(seed, k, E, S)
173     W = jax.device_put(W, jax.P('X', None, None))
174     B = jax.device_put(B, jax.P('X', None))
175     A = jax.device_put(A, jax.P('X', None))
176
177     out_naive = moe_k_experts_naive(A, W, B).block_until_ready()
178
179
180     np.testing.assert_allclose(out_shm_chunked, out_naive, rtol=1e-5, atol=1e-5)
181     print('TEST PASSED')
182
183
184 run_test(S, D, F, E, k, seed)

```

q3

```

1 import jax
2 import jax.numpy as jnp
3 import functools
4 import numpy as np
5 jax.config.update('jax_num_cpu_devices', 8)

```

part 1 (implementing AllReduce collective matmul):

```

1 """
2 A[B@X, D@Y] *_D W[D@Y, F] => Out[B@X, F]
3
4 Naive impl:
5 A[B@X, D@Y] *_D W[D@Y, F] => Out[B@X, F]{U_Y}
6 AR_Y(Out[B@X, F]{U_Y}) => Out[B@X, F]
7
8 Can't overlap comm + comp...
9
10
11 Idea:
12
13 Take simple 2x2 mesh case:
14
15 A = [[A1  A2],
16       [A3  A4]]
17
18 W = [[B1],
19       [B2]]
20
21 Out = [[O1],
22        [O2]]
23
24 Doing AW = Out
25
26 O1 = A1*B1 + A2*B2 (hence all reduce)
27

```

```

28 want to tile this further to try to overlap comm + comp
29
30 let B1 = [b_11  b_12]
31     B2 = [b_21  b_22]
32
33
34 then:
35
36 O1 = A1 * [b_11  b_12] + A2 * [b_21  b_22] = [A1*b_11  A1*b_12] + [A2*b_21  A2*
    b_22] =
37     [(A1*b_11 + A2*b_21)  (A1*b_12 + A2*b_22)]
38
39 core algorithm idea:
40
41 chunk the F dimension of W up and loop over the chunks on each device:
42 - Do local matmul with local A shard
43 - AllReduce that small chunk
44 - Add to an array and concat them
45 """
46
47 @jax.jit
48 @jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X',
    None))
49 def collective_matmul_all_reduce(A, W):
50     CHUNK_SIZE = 1024
51     def f(i, out):
52         chunk_W = jax.lax.dynamic_slice_in_dim(W, CHUNK_SIZE * i, CHUNK_SIZE, axis
            =-1)
53         out_chunk = jnp.einsum('bd,dc->bc', A, chunk_W)
54         out_chunk = jax.lax.psum(out_chunk, 'Y')
55         out = jax.lax.dynamic_update_slice_in_dim(out, out_chunk, CHUNK_SIZE * i,
            axis=-1)
56         return out
57
58     out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
59     out = jax.lax.pcast(out, 'X', to='varying')
60     out = jax.lax.fori_loop(0, (W.shape[1] + CHUNK_SIZE - 1) // CHUNK_SIZE, f, out
        , unroll=True)
61
62     return out
63
64
65 @jax.jit
66 def matmul(A, W):
67     return jnp.einsum('bd,df->bf', A, W, out_sharding=jax.P('X', None))
68
69
70 Explicit = jax.sharding.AxisType.Explicit
71 mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
72 jax.set_mesh(mesh)
73
74 B = 1024
75 D = 2048
76 F = 8192
77
78 A = jnp.arange(B * D).reshape((B, D))
79 W = jnp.arange(D * F).reshape((D, F))
80
81 A = jax.device_put(A, jax.P('X', 'Y'))

```

```

82 W = jax.device_put(W, jax.P('Y', None))
83
84 with jax.profiler.trace("/kaggle/working/q3_p1_tensorboard_jit"):
85     out_baseline = matmul(A, W).block_until_ready()
86
87 """
88 Set chunk size large enough to be out of latency bound regime
89 """
90
91 with jax.profiler.trace("/kaggle/working/q3_p1_tensorboard_collective_matmul"):
92     out_coll_matmul = collective_matmul_all_reduce(A, W).block_until_ready()
93
94 np.testing.assert_allclose(out_coll_matmul, out_baseline)

```

The jit version takes around 517 microseconds whereas the collective matmul version takes around 550 microseconds. Looking at the profile, this seems to be because the xla compiler isn't overlapping the chunked matmul fusions and ARs together. This ends up taking the same time plus the additional overhead of launching many smaller ops.

part 2 (implementing a RS collective matmul):

```

1  """
2  Tmp[B@X, F@Y] *_F W2[F@Y, D] => Out[B@X, D@Y]
3
4  Naive impl:
5  Tmp[B@X, F@Y] *_F W2[F@Y, D] => Out[B@X, D]{U_Y}
6  RS_Y,D(Out[B@X, D]{U_Y}) => Out[B@X, D@Y]
7  (again can't overlap comm and comp)
8
9  Idea:
10
11  Take simple 2x2 mesh case:
12
13  A = [[A11  A12],
14       [A21  A22]]
15
16  B = [[B11  B12],
17       [B21  B22]]
18
19  Out = [[O11  O12],
20         [O21  O22]]
21
22  Note: d# notation refers to what device that shard is on
23
24  d1(O11) = d1(A11*B11) + d2(A12*B21)
25  d2(O12) = d1(A11*B12) + d2(A12*B22)
26  d3(O21) = d3(A21*B11) + d4(A22*B21)
27  d4(O22) = d3(A21*B12) + d4(A22*B22)
28
29
30
31  observations:
32  - each A shard only needs to multiply with it's corresponding y axis index B shard
33    as it and the result permute through
34
35  core idea:

```

```

36 do local matmuls where chunk from W matrix corresponds to what y axis and shift
    the accumulator.
37 note: we must start by doing the local matmul for the next device's shard to have
    one less comm than comp.
38     this way, every comm is overlapped by a comp
39
40 we get on first step (only looking at first row of output, since the pattern can
    be used for other rows as well):
41 O11 = d2(A12*B21)
42 O12 = d1(A11*B12)
43
44 O12 needs d2(A12*B22)
45 O11 needs d1(A11*B11)
46
47 move accumulator back to corresponding shard, and update accum. we needed 2 local
    matmuls but only 1 comp.
48 """
49
50 @jax.jit
51 @jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X',
    'Y'))
52 def collective_matmul_reduce_scatter(A, W):
53     axis_size = jax.lax.axis_size('Y')
54     idx = jax.lax.axis_index('Y')
55     chunk_size = W.shape[1] // axis_size
56
57     def f(i, acc):
58         W_chunk = jax.lax.dynamic_slice_in_dim(
59             W,
60             chunk_size * ((idx + 1 + i) % axis_size),
61             chunk_size,
62             axis=1
63         )
64         local_res = jnp.einsum('bf,fc->bc', A, W_chunk)
65         acc += local_res
66
67         return jax.lax.ppermute(
68             acc,
69             'Y',
70             [(i, (i-1)%axis_size) for i in range(axis_size)]
71         )
72
73
74 out = jnp.zeros((A.shape[0], chunk_size), dtype=jnp.int32)
75 out = jax.lax.pcast(out, ('X', 'Y'), to='varying')
76 # out = jax.lax.pvary(out, ('X', 'Y'))
77 last_acc = jax.lax.fori_loop(0, axis_size - 1, f, out, unroll=True)
78
79 # last local matmul doesn't require another ppermute
80 last_W_chunk = jax.lax.dynamic_slice_in_dim(
81     W,
82     chunk_size * ((idx + axis_size) % axis_size),
83     chunk_size,
84     axis=1
85 )
86 last_acc += jnp.einsum('bf,fc->bc', A, last_W_chunk)
87 return last_acc
88
89

```

```

90 @jax.jit
91 def matmul(A, W):
92     return jnp.einsum(
93         'bf,fd->bd',
94         A,
95         W,
96         out_sharding=jax.P('X', 'Y')
97     )
98
99
100 Explicit = jax.sharding.AxisType.Explicit
101 mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
102 jax.set_mesh(mesh)
103
104 B = 1024
105 D = 2048
106 F = 8192
107
108 A = jnp.arange(B*F).reshape((B, F))
109 W = jnp.arange(F*D).reshape((F, D))
110
111 A = jax.device_put(A, jax.P('X', 'Y'))
112 W = jax.device_put(W, jax.P('Y', None))
113
114 with jax.profiler.trace("/kaggle/working/q3_p2_tensorboard_collective_matmul"):
115     out = collective_matmul_reduce_scatter(A, W).block_until_ready()
116
117 with jax.profiler.trace("/kaggle/working/q3_p2_tensorboard_jit"):
118     out_baseline = matmul(A, W).block_until_ready()
119
120 np.testing.assert_array_equal(out, out_baseline)
121 print('TEST PASSED')

```

The naive jit version takes around 300us whereas the collective all reduce version takes 280us.

part 3 (putting them together into an end-to-end Transformer block):

```

1 @jax.jit
2 @jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P(None, 'Y')), out_specs=jax.P('X',
    'Y'))
3 def collective_matmul_all_gather(A, W):
4     """
5     core idea:
6     rotate A chunks and index out corresponding W chunk. Keep running accumulator
7     and final result is already on the device
8     """
9     axis_size = jax.lax.axis_size('Y')
10    idx = jax.lax.axis_index('Y')
11    chunk_size = A.shape[1]
12
13    def f(i, carry):
14        out, A_chunk = carry
15        W_chunk = jax.lax.dynamic_slice_in_dim(W, chunk_size * ((idx - i) %
16            axis_size), chunk_size)
17        out += jnp.einsum('bd,df->bf', A_chunk, W_chunk)
18        A_chunk = jax.lax.ppermute(A_chunk, 'Y', [(i, (i+1)%axis_size) for i in
19            range(axis_size)])
20    return out, A_chunk

```

```

18 out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
19 out = jax.lax.pcast(out, ('X', 'Y'), to='varying')
20 # out = jax.lax.pvary(out, ('X', 'Y'))
21 out, last_A_chunk = jax.lax.fori_loop(0, axis_size - 1, f, (out, A), unroll=
22     True)
23
24 # do last local matmul independently so we don't do extra comm
25 last_W_chunk = jax.lax.dynamic_slice_in_dim(W, chunk_size * ((idx - (axis_size
26     - 1)) % axis_size), chunk_size)
27 out += jnp.einsum('bd,df->bf', last_A_chunk, last_W_chunk)
28 return out
29
30 @functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
31 def matmul(A, W):
32     return A @ W
33
34
35 Explicit = jax.sharding.AxisType.Explicit
36 mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
37 jax.set_mesh(mesh)
38
39 B = 1024
40 D = 2048
41 F = 8192
42
43 A = jnp.arange(B*D).reshape((B, D))
44 W = jnp.arange(F*D).reshape((D, F))
45
46 A = jax.device_put(A, jax.P('X', 'Y'))
47 W = jax.device_put(W, jax.P(None, 'Y'))
48
49 with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_ag_collective_matmul"):
50     out = collective_matmul_all_gather(A, W)
51
52 with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_ag_jit"):
53     out_baseline = matmul(A, W)
54
55 np.testing.assert_array_equal(out, out_baseline)
56 print("TEST PASSED")

```

```

1 """
2 In[B@X, D@Y] *_D W_in[D, F@Y] *_F W_out[F@Y, D]
3 """
4
5 @jax.jit
6 def collective_mlp_block(In, W1, W2):
7     tmp = collective_matmul_all_gather(In, W1)
8     return collective_matmul_reduce_scatter(tmp, W2)
9
10
11 @functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
12 def mlp_block(In, W1, W2):
13     tmp = jnp.einsum('bd,df->bf', In, W1, out_sharding=jax.P('X', 'Y'))
14     return jnp.einsum('bf,fd->bd', tmp, W2, out_sharding=jax.P('X', 'Y'))
15
16
17 B = 1024

```

```

18 D = 2048
19 F = 8192
20
21 In = jnp.arange(B*D).reshape((B, D))
22 W1 = jnp.arange(D*F).reshape((D, F))
23 W2 = jnp.arange(F*D).reshape((F, D))
24
25 In = jax.device_put(In, jax.P('X', 'Y'))
26 W1 = jax.device_put(W1, jax.P(None, 'Y'))
27 W2 = jax.device_put(W2, jax.P('Y', None))
28
29 with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_collective_mlp"):
30     out = collective_mlp_block(In, W1, W2)
31
32 with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_mlp_jit"):
33     out_baseline = mlp_block(In, W1, W2)
34
35 np.testing.assert_array_equal(out, out_baseline)
36 print('TEST PASSED')

```

The collective matmul version took 510us whereas the jit version took 585us.

q4

```

1 import jax
2 import jax.numpy as jnp
3 import functools
4 import numpy as np
5 # jax.config.update('jax_num_cpu_devices', 8)

```

```

1 """
2 core change to make it bidirectional:
3 split the one big all to all into 2 smaller alltoalls and pray that xla compiles
   bidirectionally
4 """
5
6
7 @jax.jit
8 @jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X',
   None))
9 def collective_matmul_all_reduce_bidir(A, W):
10     CHUNK_SIZE = 1024
11     def f(i, out):
12         chunk_W = jax.lax.dynamic_slice_in_dim(W, CHUNK_SIZE * i, CHUNK_SIZE, axis
   =-1)
13         local_res = jnp.einsum('bd,dc->bc', A, chunk_W)
14         out_subchunk_1 = jax.lax.psum(
15             jax.lax.dynamic_slice_in_dim(local_res, 0, CHUNK_SIZE // 2, axis=-1),
16             'Y'
17         )
18         out_subchunk_2 = jax.lax.psum(
19             jax.lax.dynamic_slice_in_dim(local_res, CHUNK_SIZE // 2, CHUNK_SIZE //
20             2, axis=-1),
21             'Y'
22         )
23         out = jax.lax.dynamic_update_slice_in_dim(
24             out,

```



```

24         jnp.hstack([out_subchunk_1, out_subchunk_2]),
25         CHUNK_SIZE * i,
26         axis=-1
27     )
28     return out
29
30     out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
31     out = jax.lax.pcast(out, 'X', to='varying')
32     # out = jax.lax.pvary(out, 'X')
33     out = jax.lax.fori_loop(0, (W.shape[1] + CHUNK_SIZE - 1) // CHUNK_SIZE, f, out
34         , unroll=True)
35
36     return out
37
38 Explicit = jax.sharding.AxisType.Explicit
39 mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
40 jax.set_mesh(mesh)
41
42 B = 1024
43 D = 2048
44 F = 8192
45
46 A = jnp.arange(B * D).reshape((B, D))
47 W = jnp.arange(D * F).reshape((D, F))
48
49 A = jax.device_put(A, jax.P('X', 'Y'))
50 W = jax.device_put(W, jax.P('Y', None))
51
52 with jax.profiler.trace("/kaggle/working/q4_tensorboard_unidir_ar_matmul"):
53     out_coll_matmul_unidir = collective_matmul_all_reduce(A, W).block_until_ready()
54
55 """
56 Set chunk size large enough to be out of latency bound regime
57 """
58
59 with jax.profiler.trace("/kaggle/working/q4_tensorboard_bidir_ar_matmul"):
60     out_coll_matmul_bidir = collective_matmul_all_reduce_bidir(A, W).
61         block_until_ready()
62
63 np.testing.assert_array_equal(out_coll_matmul_unidir, out_coll_matmul_bidir)
64 print("TEST PASSED")

```

The unidirectional AllReduce collective matmul took 561us whereas the bidirectional AllReduce collective matmul took 590us. Looking at the trace, the xla compiler doesn't seem to fuse the two AllReduce's and do them bidirectionally.

```

1 """
2 idea:
3
4 split the A chunk into two halves (split by rows), move on left and one right
5 and get the corresponding slices of the W matrix as you process them.
6
7 indexing changes depending on the direction you permute chunk but reasoning is
8     similar to
9     unidirectional case.
10 """

```

```

10
11
12 @jax.jit
13 @jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X',
    'Y'))
14 def collective_matmul_reduce_scatter_bidir(A, W):
15     axis_size = jax.lax.axis_size('Y')
16     idx = jax.lax.axis_index('Y')
17     chunk_size = W.shape[1] // axis_size
18     top_A_slice = jax.lax.dynamic_slice_in_dim(A, 0, A.shape[0] // 2)
19     bottom_A_slice = jax.lax.dynamic_slice_in_dim(A, A.shape[0] // 2, A.shape[0] -
        (A.shape[0] // 2))
20
21     def f(i, carry):
22         top_acc, bottom_acc = carry
23         top_W_chunk = jax.lax.dynamic_slice_in_dim(
24             W,
25             chunk_size * ((idx + 1 + i) % axis_size),
26             chunk_size,
27             axis=1
28         )
29         top_local_res = jnp.einsum('bf,fc->bc', top_A_slice, top_W_chunk)
30         top_acc += top_local_res
31         top_acc = jax.lax.ppermute(
32             top_acc,
33             'Y',
34             [(i, (i-1)%axis_size) for i in range(axis_size)]
35         )
36
37         bottom_W_chunk = jax.lax.dynamic_slice_in_dim(
38             W,
39             chunk_size * ((idx - 1 - i) % axis_size),
40             chunk_size,
41             axis=1
42         )
43         bottom_local_res = jnp.einsum('bf,fc->bc', bottom_A_slice, bottom_W_chunk)
44         bottom_acc += bottom_local_res
45         bottom_acc = jax.lax.ppermute(
46             bottom_acc,
47             'Y',
48             [(i, (i+1)%axis_size) for i in range(axis_size)]
49         )
50
51         return top_acc, bottom_acc
52
53
54 top_acc = jnp.zeros((top_A_slice.shape[0], chunk_size), dtype=jnp.int32)
55 top_acc = jax.lax.pcast(top_acc, ('X', 'Y'), to='varying')
56 # top_acc = jax.lax.pvary(top_acc, ('X', 'Y'))
57 bottom_acc = jnp.zeros((bottom_A_slice.shape[0], chunk_size), dtype=jnp.int32)
58 bottom_acc = jax.lax.pcast(bottom_acc, ('X', 'Y'), to='varying')
59 # bottom_acc = jax.lax.pvary(bottom_acc, ('X', 'Y'))
60
61 top_acc, bottom_acc = jax.lax.fori_loop(0, axis_size - 1, f, (top_acc,
    bottom_acc), unroll=True)
62
63 # last local matmul doesn't require another ppermute
64 last_W_chunk = jax.lax.dynamic_slice_in_dim(
65     W,

```

```

66         chunk_size * ((idx + axis_size) % axis_size),
67         chunk_size,
68         axis=1
69     )
70     top_acc += jnp.einsum('bf,fc->bc', top_A_slice, last_W_chunk)
71     bottom_acc += jnp.einsum('bf,fc->bc', bottom_A_slice, last_W_chunk)
72     return jnp.vstack([top_acc, bottom_acc])
73
74
75 @jax.jit
76 def matmul(A, W):
77     return jnp.einsum(
78         'bf,fd->bd',
79         A,
80         W,
81         out_sharding=jax.P('X', 'Y')
82     )
83
84
85 Explicit = jax.sharding.AxisType.Explicit
86 mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
87 jax.set_mesh(mesh)
88
89 B = 1024
90 D = 2048
91 F = 8192
92
93 A = jnp.arange(B*F).reshape((B, F))
94 W = jnp.arange(F*D).reshape((F, D))
95
96 A = jax.device_put(A, jax.P('X', 'Y'))
97 W = jax.device_put(W, jax.P('Y', None))
98
99 with jax.profiler.trace("/kaggle/working/q4_tensorboard_unidir_rs_matmul"):
100     out_unidir = collective_matmul_reduce_scatter(A, W).block_until_ready()
101
102 with jax.profiler.trace("/kaggle/working/q4_tensorboard_bidir_rs_matmul"):
103     out_bidir = collective_matmul_reduce_scatter_bidir(A, W).block_until_ready()
104
105 np.testing.assert_array_equal(out_unidir, out_bidir)
106 print('TEST PASSED')

```

The unidirectional ReduceScatter collective matmul took 280us whereas the bidirectional ReduceScatter collective matmul took 250us.